CSE325 | 19-20 | A

Ans. to ques no. 1

1. (a) Describe briefly how Unified Modeling Language is used for visualizing, specifying, constructing and documenting.

Unified Modeling Language (UML) is a standardized visual modeling language used to describe, design, and document software systems. It gives "pictures or views" of an object-oriented system, which helps people understand the system better. UML is useful because "a picture is worth a thousand words."

Visualizing

UML helps developers, analysts, clients, and other stakeholders **see the system before it is built**. Instead of only writing code or long descriptions, UML diagrams show the system's functions, users, classes, objects, and interactions visually.

For example, in a microwave oven system, a use case diagram can visually show that the **User** interacts with use cases such as **Set Timer**, **Start Heating**, **Stop Heating**, and **Receive Completion Beep**.

Specifying

UML helps specify **what the system should do** and how different parts are related. Use case diagrams specify the functional requirements of the system from the user's point of view. The slide says use case diagrams describe **what a system does rather than how it does it**.

For the microwave oven, UML specifies that the user must place food, set the timer, press start, and may stop heating at any time.

Constructing

UML supports construction because developers can use diagrams as a blueprint before coding. The slide says UML supports implementation and testing because teams can visualize processes, user interactions, and the static structure of the system.

For example, after identifying use cases like **Set Timer** and **Heat Food**, programmers can implement keypad input, timer control, heating control, stop button behavior, and beep signal.

Documenting

UML is also used to document the system for future maintenance and communication. Class diagrams, for example, document the structure of a system by showing classes, attributes, methods, and relationships between classes.

So, UML becomes a permanent record of the system design. New developers can later understand the microwave oven system without reading the entire code first.

(b) Consider a microwave oven that uses microwave technology for the purpose of heating foods. If you want to heat food items you must enter the food item in the heating chamber and then set the timer using the key pad in the front panel located on the door of micro wave oven. You have to press the start button to start heating. The oven will give a beep signal when the heating is complete. You can stop heating by pressing the Stop button any time. Do the following for the micro wave oven system: **(6+**

(i) Draw the use case diagram by identifying the actors and use cases for this micro wave oven system.

(ii) Write down the detailed use case scenarios related to setting the timer and heating.

(iii) Write down the acceptance test cases for these use cases as well.

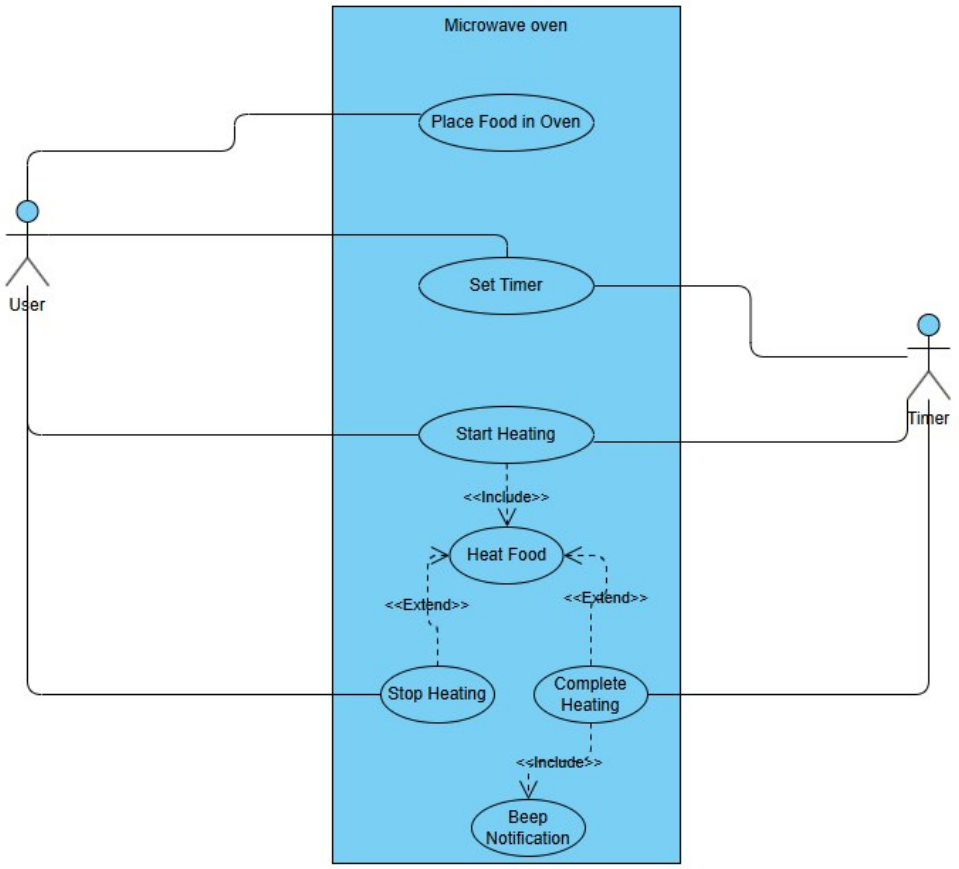
i)

Actors:

Primary - User
Secondary - Timer

Use Cases:

1. Place Food in Oven
2. Set Timer
3. Start Heating
4. Heat Food
5. Stop Heating
6. Complete Heating
7. Beep Notification



Ans. to the ques no. 2

2. (a) Explain the concept for Class and Instance with necessary examples.

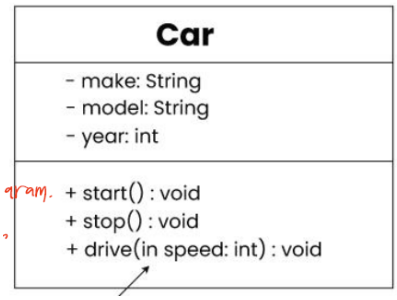
Class

A **class** is a blueprint or template for objects. In UML class diagrams, a class is shown as a rectangle with three parts:

1. **Class name** at the top
2. **Attributes** in the middle
3. **Operations/methods** at the bottom

Your slide says a UML class diagram represents the structure of a system by showing **classes, attributes, methods, and relationships/connections between classes.**

Here, **Car** is a class because it defines common attributes and operations for cars.



Instance

An **instance** is a specific object created from a class. The class is general, but the instance is real and specific.

Example:

Class: ATM

Instances:

atm1 = ATM(location = "Dhaka Branch")

atm2 = ATM(location = "Gazipur Branch")

So, **ATM** is the class, and atm1, atm2 are instances.

(b) An account holder of Sonali Bank can withdraw an amount from his or her account through debit card with the help of ATM (Automatic Teller Machine) booth. The account holder must insert the card into the machine and then after authorization through PIN (Personal Identification Number) the amount of money to be withdrawn will be entered through keypad of the ATM machine. The requested amount will be checked against the balance of the bank account and the amount will be dispensed from the machine if there is available balance to satisfy the request. There must be a notification to the registered phone number to the account holder from central server of the bank. Finally, the ATM machine ejects the card after the transactions. Show the steps of identifying the classes from this use case scenario.

****You can skip writing details under each step and go straight to class diagram****

Step 1: Read the scenario and identify important nouns

Account Holder, Sonali Bank, Bank Account, Debit Card, ATM Booth / ATM Machine, PIN, Amount, Keypad, Balance, Notification, Registered Phone Number ,Central Server, Transaction, Cash / Money

Step 2: Remove simple data values

Some nouns are not separate classes; they can be attributes.

- PIN (PIN belongs to Debit Card or Account Holder authentication)
- Amount (It is data used in withdrawal transaction)
- Balance
- Phone Number
- Cash

Step 3: Select candidate classes

AccountHolder
Bank
BankAccount
DebitCard
ATM
Keypad
CentralServer
Transaction
Notification

These are selected because they are meaningful objects in the system.

Step 4: Identify attributes of each class

- **AccountHolder:** name, phoneNo
- **Bank:** name, code
- **BankAccount:** accountNo, balance
- **DebitCard:** cardNo, pin
- **ATM:** atmId, location, availableCash
- **Keypad:** keypadId
- **CentralServer:** serverId
- **Transaction:** transactionId, amount, date, type
- **Notification:** notificationId, message, phoneNo

Step 5: Identify operations/functions of each class

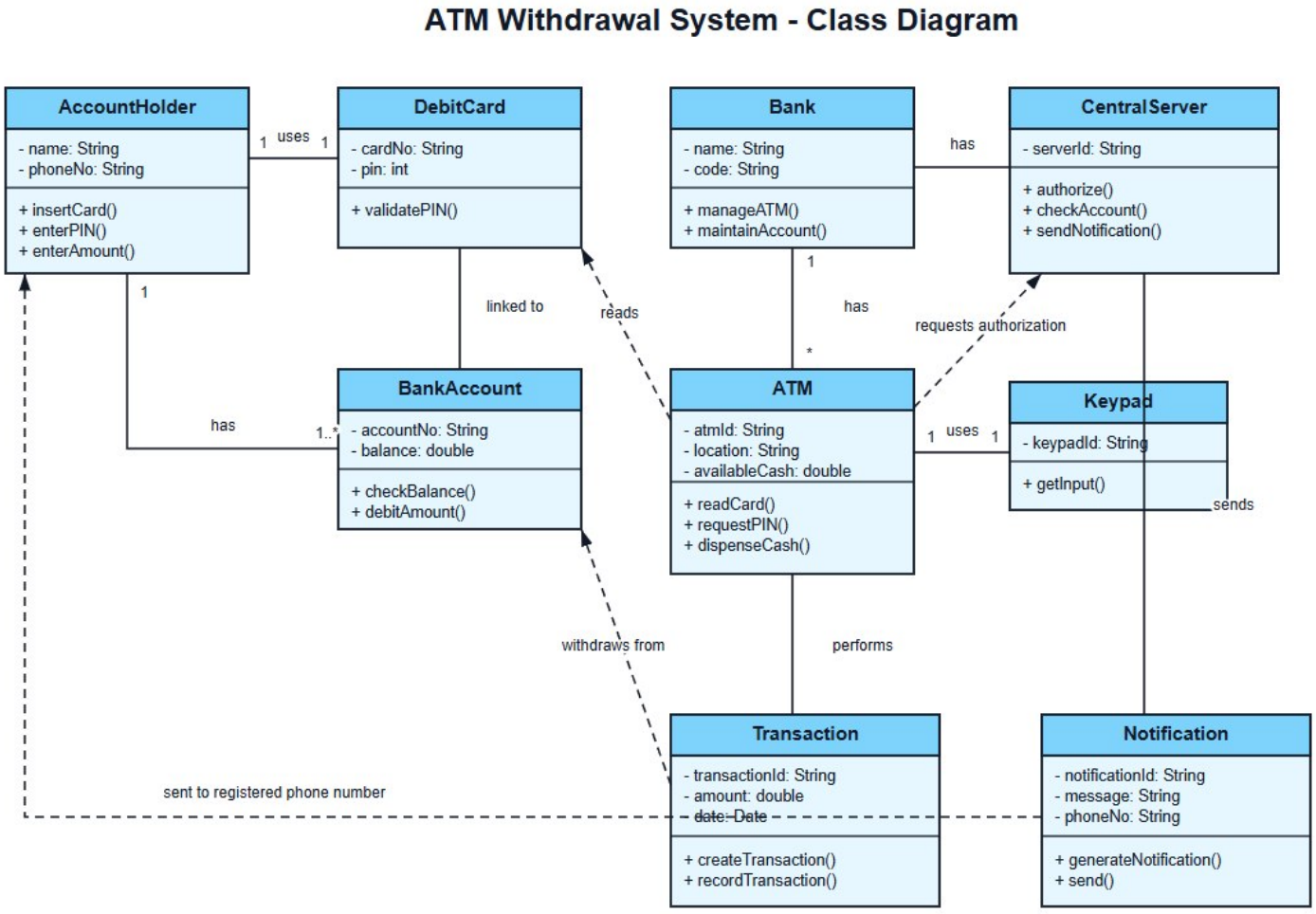
- **AccountHolder:** insertCard(), enterPIN(), enterAmount()
- **DebitCard:** validatePIN()
- **ATM:** readCard(), requestPIN(), dispenseCash(), ejectCard()
- **Keypad:** getInput()
- **BankAccount:** checkBalance(), debitAmount()
- **CentralServer:** authorize(), checkAccount(), sendNotification()
- **Transaction:** createTransaction(), recordTransaction()
- **Notification:** generateNotification(), send()

Step 6: Identify relationships between classes

Relationships in this scenario

AccountHolder has BankAccount
AccountHolder uses DebitCard
Bank has CentralServer
Bank has ATM
ATM uses Keypad
ATM reads DebitCard
ATM performs Transaction
Transaction withdraws amount from BankAccount
CentralServer authorizes DebitCard / PIN
CentralServer sends Notification
Notification is sent to AccountHolder
ATM ejects DebitCard

Step 7: Draw diagram



2(c), 3(a, b, c, d), 4(a, c)

Syllabus-ई नाहीं

4. (b) What are the advantages of Distributed Version Control model of source control over Copy Modify Merge Model? Consider a scenario where Andy, Bobby, Cally clone the remote repository locally. The work locally on a certain File F and commit locally to their repositories. What will happen if Cally wants to Push their change first, then Andy tries to push his change and finally Bobby tries to Push her change. Show the steps for conflict resolution in this scenario.

Advantages of Distributed Version Control over Copy-Modify-Merge model (Centralized Version Control)

In a **Distributed Version Control System**, every developer has a **local repository** along with the working copy. The slide shows that in Distributed VCS, developers can **commit/update locally** and then use **push/pull** to communicate with the remote repository.

Compared to the CVCS model, Distributed VCS has these advantages:

1. **Each developer has a full local repository**

   Andy, Bobby, and Cally do not only have a working copy. Each of them has their own local repository. So they can work and commit locally before sending changes to the remote repository.

2. **Local commits are possible**

   In Distributed VCS, developers can commit changes to their own local repository first. They do not need to immediately update the central/remote repository.

3. **Better history tracking**

   Version control maintains a complete history of who changed what, when, and why. So the project has a clear commit history instead of only file copies.

4. **Push and pull support collaboration**

   Developers can **push** their local commits to the remote repository and **pull** changes from the remote repository when needed.

5. **Conflicts are resolved locally**

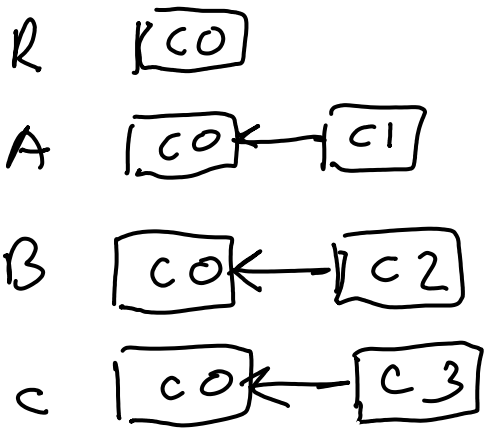
   If the remote repository has already changed, a developer's push may fail. Then the developer pulls the remote changes, merges them locally, resolves conflicts locally, and pushes again.

6. **Rollback and reliability**

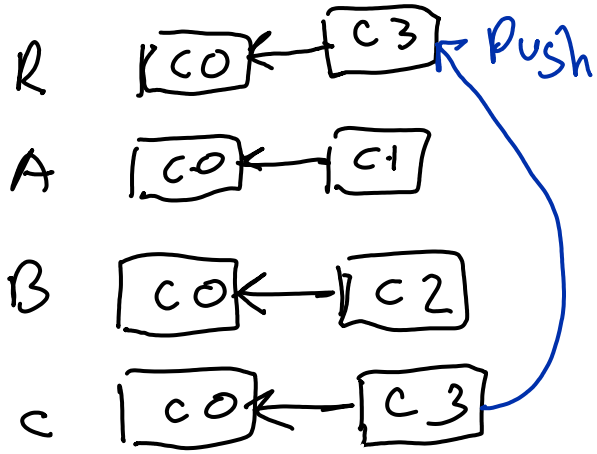
   Version Control Systems allow rollback to previous stable versions and improve accountability, reproducibility, and reliability.

Remote = R, Andy = A, Bobby = B, Cally = C

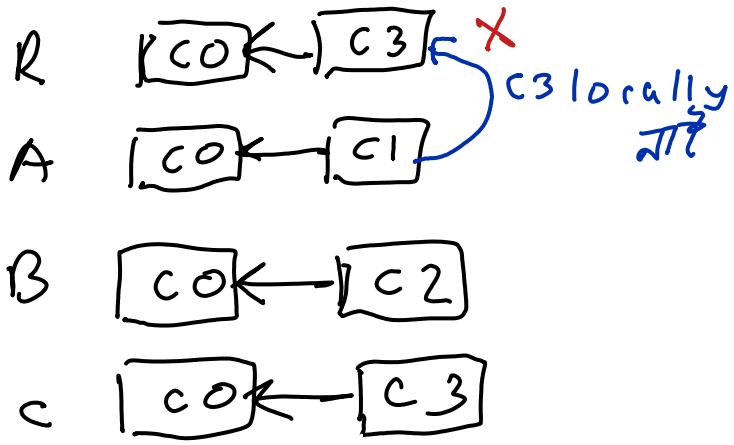
After initial commits:



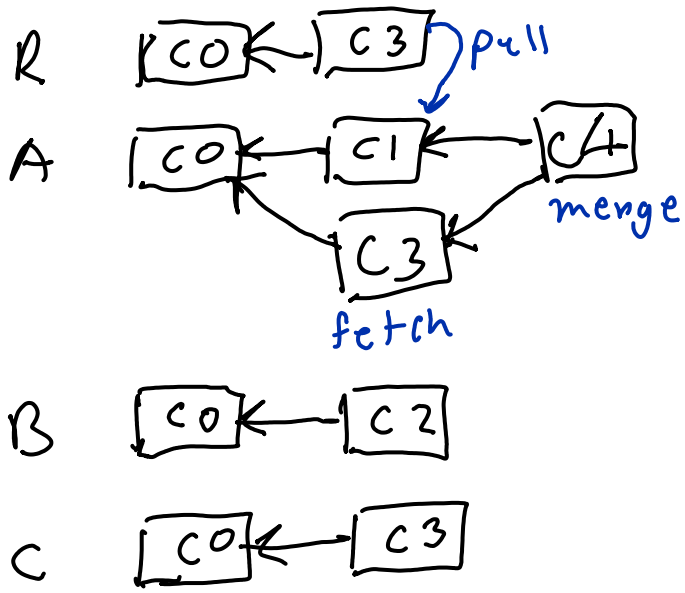
Callie pushes:



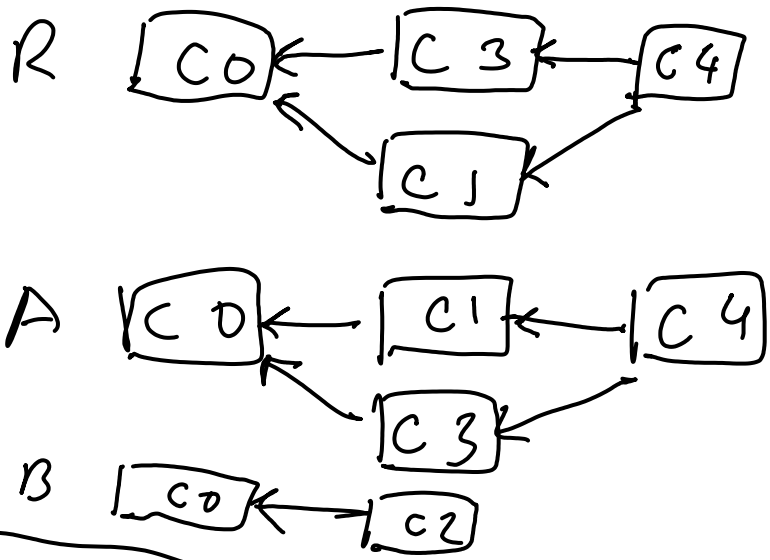
Andy tries to push:



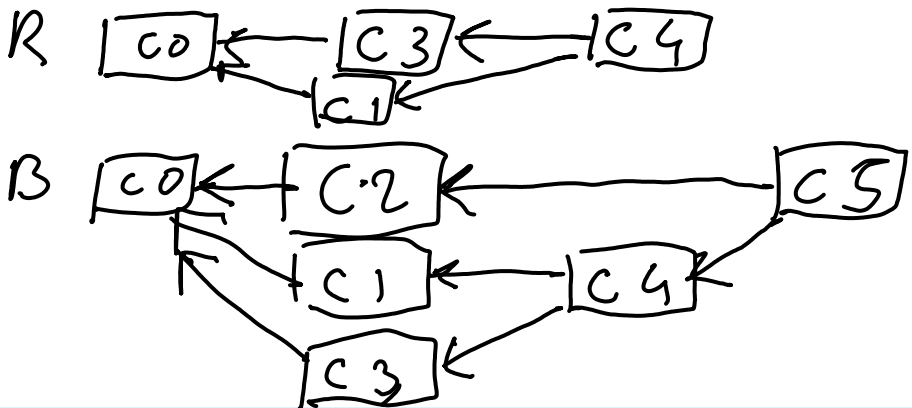
Andy pulls:



Andy pushes



Bobby pulls



Bobby pushes: